

Real-Time Search Index Compression

Given Recurrence

$$T(n) = T(n/2) + \log n \quad T(n) = T(n/2) + \log n \quad T(n) = T(n/2) + \log n$$

Substitution Method (Expansion)

Expand the recurrence:

$$\begin{aligned} T(n) &= T(n/2) + \log n \\ T(n) &= T(n/2) + \log n \\ T(n) &= T(n/2) + \log n \\ &= T(n/4) + \log(n/2) + \log n \\ &= T(n/4) + \log(n/2) + \log n \\ &= T(n/4) + \log(n/2) + \log n \\ &= T(n/8) + \log(n/4) + \log(n/2) + \log n \\ &= T(n/8) + \log(n/4) + \log(n/2) + \log n \\ &= T(n/8) + \log(n/4) + \log(n/2) + \log n \end{aligned}$$

After k expansions,

$$\begin{aligned} T(n) &= T(n/2^k) + \sum_{i=0}^{k-1} \log(n/2^i) \\ T(n) &= T(n/2^k) + \sum_{i=0}^{k-1} (\log n - \log 2^i) \\ T(n) &= T(n/2^k) + \sum_{i=0}^{k-1} (\log n - i) \end{aligned}$$

The recursion stops when

$$n/2^k = 1 \quad \frac{n}{2^k} = 1 \quad 2^k = n$$

Therefore,

$$k = \log_2 n \quad 2^k = n \quad k = \log_2 n$$

Each recursive level contributes $O(\log n)$ work, and there are $O(\log n)$ levels.

Hence,

$$\begin{aligned} T(n) &= O(\log n \times \log n) \\ T(n) &= O(\log n \times \log n) \\ T(n) &= O(\log n \times \log n) \\ T(n) &= O((\log n)^2) \end{aligned}$$

Recursion Depth Analysis

- Input size becomes half at every recursive call.
 - Number of recursive levels = $\log_2 n$.
 - Work at each level is proportional to $\log n$.
 - Total work = $\log n \times \log n$.
-

Asymptotic Time Complexity

$O((\log n)^2)$ $O((\log n)^2)$ $O((\log n)^2)$

Source Code

```
import math

def search_index_compression(n):
    if n <= 1:
        return 0
    return math.log2(n) + search_index_compression(n // 2)

n = int(input("Enter the input size: "))

result = search_index_compression(n)

print("Total indexing work:", round(result, 2))
print("Asymptotic Time Complexity: O((log n)^2)")
```

Output

```
 #Question 1 Application: Real-Time Search Index Compression
import math

def search_index_compression(n):
    if n <= 1:
        return 0
    return math.log2(n) + search_index_compression(n // 2)

n = int(input("Enter the input size: "))

result = search_index_compression(n)

print("Total indexing work:", round(result, 2))
print("Asymptotic Time Complexity: O((log n)^2)")

... Enter the input size: 16
Total indexing work: 10.0
Asymptotic Time Complexity: O((log n)^2)
```